HW2 回歸作業規劃

10/16/2026 10:11:07
https://chatgpt.com/g/g-p-6ab3b2b9eaa88191b06dd4d134b22f16-aiot-nchu/c/6acef1b1-44fc-83e8-bce4-ed1d271e023d

HW2: Multiple Linear regression **說明** **一、作業目標：**\ 本次作...

Thinking ● 已搜尋 1 個網站 ✓ 思考兩秒鐘 我看完這份 HW2 規格後，這份作業...

根據上面規劃，以及原先的HW2作業要求後，After grill me and providing me...

可以。我會先從「老師會怎麼挑毛病」的角度拷問這個 HW2，不只確認能不能跑...

help me write skilearn to solve Student Productivity kaggle - and allow...

我會直接把它設計成一份可交作業的 sklearn + CRISP-DM Notebook 架構，而...

provide me the program

下面給你一份可以直接當作 7114056XXX_hw2.py 使用的完整版本。它包含...

HW2: Multiple Linear regression

說明

一、作業目標：\
本次作業延伸自 HW1，目標是讓同學能夠以實際資料集進行「多元線性回歸 (Multiple Linear Regression)」的完整分析，並遵循 CRISP-DM 流程完成從資料理解、建模到評估的全過程。\

二、作業內容：

1\. 資料來源

  至 Kaggle 選擇一個具有 10 至 20 個特徵 (features) 的公開資料集。\
  類型不限（可為房價預測、醫療、車輛效能等主題）。\
  請明確標示資料集來源與連結。

2\. 分析任務\
  使用線性回歸 (Linear Regression) 模型進行預測。\
  可嘗試單純線性回歸、多元線性回歸或 Auto Regression。\
  必須執行** 特徵選擇 (Feature Selection) **與 **模型評估 (Model Evaluation)**。\
  結果部分需包含請提供預測圖(加上信賴區間或預測區間)

3\. CRISP-DM 流程說明\
    Business Understanding\
    Data Understanding\
    Data Preparation\
    Modeling\
    Evaluation\
    Deployment

\
4\. AI協助要求

所有與 ChatGPT 的對話請以 pdfCrowd 或其他方式須**匯出為 PDF**\
請**使用 NotebookLM** 對網路上同主題的解法進行研究，並撰寫一份 100 字以上的摘要，放入報告中。\
請在報告中明確標示「GPT 輔助內容」與「NotebookLM 摘要」

5\. 繳交內容\
主程式：7114056XXX_hw2.py/.ipynb\
報告檔： PDF，需包含以下內容：\
按照 CRISP-DM 說明的分析流程\
GPT 對話過程（pdfCrowd 匯出）\
NotebookLM 研究摘要\
網路上主流或更優解法之比較與說明\
以上檔案與資料夾**請壓縮為學號命名的一個zip**（例如 7114056XXX_hw2.zip）上傳。\
\
(optional) 若上傳至 GitHub，或是以colab撰寫，需提供連結，並在 README.md 中整理流程與成果。

\
三、評分標準

** 文件說明（50%）**\
CRISP-DM 流程完整且邏輯清楚（25%）\
包含 GPT 對話與 NotebookLM 摘要（15%）\
有明確說明資料集來源與研究脈絡（10%）
\
** 結果呈現（50%）**\
模型正確可執行，具特徵選擇與評估（25%）\
結果合理、美觀且具有說服力（15%）\
呈現出Kaggle名次(若有)/預測結果評估(預測圖、評估指標)（10%）

2026/10/14 上午11:07:03

Thinking

- **已搜尋 1 個網站**

✓ 思考兩秒鐘

我看完這份 HW2 規格後，這份作業其實不是單純「跑一個 `LinearRegression()`」而已，而是要做成一個**小型、完整的 Regression Data Science Project**。而且老師把評分拆成「文件 50% + 結果 50%」，所以資料集選擇、CRISP-DM 脈絡、Feature Selection、模型比較、Prediction Interval 都很重要。

我會建議我們不要急著寫程式，先把整份作業的「戰略」定下來。

我最推薦的題目：學生生產力預測

目前 Kaggle 有一個很適合這份 HW2 的 **Student Productivity Prediction Challenge**，目標是利用學生的生活與學習行為預測 `productivity_score`。它有 `age`、讀書時間、睡眠、手機使用、社群媒體、YouTube、遊戲、休息次數、咖啡因、運動、作業完成數、出席率、壓力、專注度、成績等特徵，特徵數量與這次要求的 **10～20 個 features** 非常吻合。(Kaggle)

Student Productivity Prediction Challenge | Kaggle

而且它還有一個很大的優勢：這是一個真正的 Kaggle Regression Competition，官方評估指標就是 **RMSE，越低越好**，所以可以直接滿足老師要求的「Kaggle 名次（若有）/ 預測結果評估」。(Kaggle)

這比一般「下載 CSV 自己做」更容易讓報告有完整故事。

一、我們可以把 HW2 定義成這個研究問題

> ***使用 Multiple Linear Regression 預測學生 Productivity Score，並分析哪些學習習慣、生活型態與學業因素最能解釋學生生產力。***

這樣就不只是：

> *我要用 Linear Regression 預測一個數字。*

而是有一個 Business / Research Question：

> *「學生每天讀書多久、睡眠多久、手機使用多久、運動量、壓力、專注度與出席率，哪些因素真正影響 Productivity？」*

這就非常適合 CRISP-DM。

二、整份 HW2 架構

我建議最後 Notebook 做成下面這條 Pipeline：

```
Kaggle Dataset
      ↓
Business Understanding
      ↓
Data Understanding
      ↓
EDA
      ↓
Data Cleaning
      ↓
Feature Engineering
      ↓
Feature Selection
      ↓
Train / Test Split
      ↓
Baseline Multiple Linear Regression
      ↓
Feature Selected Linear Regression
      ↓
Ridge / Lasso Regression
      ↓
Model Evaluation
      ↓
Prediction Interval
      ↓
Kaggle Submission
      ↓
Deployment / Conclusion
```

其中真正的核心是：

```
原始 16 個 Features
        ↓
Correlation
        ↓
VIF
```

```
        ↓
Feature Selection
        ↓
Multiple Linear Regression
        ↓
MAE / RMSE / R²
```

這樣 Feature Selection 就不是為了符合老師要求「硬做」，而是有統計上的理由。

三、CRISP-DM 可以怎麼寫

① Business Understanding

研究問題：

> *學生的學習習慣、生活型態與學業表現可能共同影響其 Productivity。本研究希望建立 Multiple Linear Regression 模型，預測學生的 Productivity Score，並分析不同因素對生產力的影響程度。*

目標變數：

```
y = productivity_score
```

輸入：

```
age
gender
study_hours_per_day
sleep_hours
phone_usage_hours
social_media_hours
youtube_hours
gaming_hours
breaks_per_day
coffee_intake_mg
exercise_minutes
assignments_completed
attendance_percentage
stress_level
focus_score
```

```
final_grade
...
```

Kaggle 資料說明確實列出了這些學習、生活與學業特徵。(Kaggle)

四、Data Understanding

這一段不要只有：

```
df.head()
```

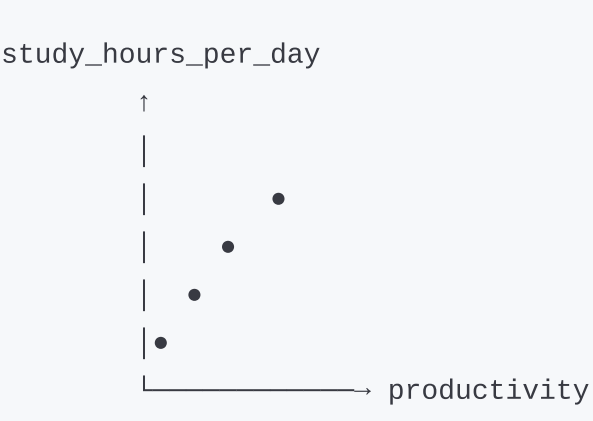
建議至少做：

```
df.shape
df.info()
df.describe()
df.isnull().sum()
df.nunique()
```

然後做 EDA：

```
Target distribution
Correlation Heatmap
Feature distributions
Feature vs Productivity
Outlier detection
Categorical feature distribution
```

例如：



我們想回答的不是只有「資料長什麼樣」，而是：

> *哪些 X 可能跟 y 有線性關係？*

因為這才跟 Linear Regression 有關。

五、Data Preparation

這部分可以讓報告看起來很完整。

例如：

```
Missing Value
      ↓
Outlier
      ↓
Categorical Encoding
      ↓
Feature Scaling
      ↓
Feature Selection
      ↓
Train/Test Split
```

Categorical feature：

```
gender
```

可以：

```
pd.get_dummies()
```

或：

```
OneHotEncoder()
```

然後 numeric features：

```
study_hours
sleep_hours
phone_usage
...
```

六、這次最重要：Feature Selection

老師明確要求：

> *必須執行 Feature Selection*

所以這一塊我建議做 **三層篩選**。

Level 1：Correlation

先看：

```
correlation_matrix = df.corr(numeric_only=True)
```

找：

```
feature → productivity_score
```

關係。

Level 2：VIF

Multiple Linear Regression 很怕：

> *Multicollinearity*

例如：

```
phone_usage_hours
social_media_hours
youtube_hours
gaming_hours
```

這些變數彼此可能高度相關。

我們可以算：

```
VIF
```

概念上：

```
VIF ≈ 1
很好

VIF 1\~5
通常可以接受

VIF > 5
需要注意

VIF > 10
嚴重 Multicollinearity
```

然後移除高度共線的 Feature。

這其實就是非常漂亮的「Feature Selection」。

Level 3：Lasso

再加入：

```
LassoCV
```

因為 Lasso 的 L1 regularization 會把不重要的 coefficient 壓到：

```
0
```

所以我們可以比較：

```
Correlation
      ↓
VIF
      ↓
Lasso
```

最後留下真正重要的 Feature。

這樣老師問：

> *你 Feature Selection 怎麼做？*

你就不是回答：

> *我把一些看起來不重要的欄位刪掉。*

而是：

> *我先透過相關係數分析 Target 關聯，再使用 VIF 排除 Multicollinearity，最後利用 Lasso Regularization 進一步進行變數篩選。*

這個說法會完整很多。

七、模型不要只跑一個

至少做三組：

Model	用途
Multiple Linear Regression	Baseline
Feature Selected Linear Regression	本作業核心
Ridge Regression	處理 Multicollinearity
Lasso Regression	Feature Selection

最後表格：

Model	Features	MAE	RMSE	R^2
Linear Regression	16	…	…	…
Selected LR	10	…	…	…
Ridge	16	…	…	…
Lasso	16	…	…	…

這張會是報告最重要的表之一。

八、Evaluation

老師要求 Model Evaluation，所以不要只有：

```
model.score()
```

至少：

MAE

平均預測錯多少。

RMSE

對「預測錯很多」的案例懲罰比較大。

而且 Kaggle 這個 competition 本身就是用 RMSE 評分。(Kaggle)

R^2

解釋能力：

```
R² = 0.80
```

可以理解成：

> *模型大約能解釋 Productivity Score 80% 的變異。*

九、預測圖

老師特別要求：

> *請提供預測圖（加上信賴區間或預測區間）*

所以這裡不能只畫：

```
Actual vs Predicted
```

基本圖可以是：

```
Predicted
   ↑
```

```
100|             ●
 90|          ●
 80|       ●
 70|    ●
 60| ●
   └──────────────────→ Actual
      60 70 80 90 100
```

理想狀況就是：

```
y = x
```

附近。

但我們再多做一張：

```
Prediction
|
|         ┌──────── Upper 95%
|       / /
|      / ●
|     / /
|----●------------- Prediction
|   / /
|  ● /
| / /
|──────── Lower 95%
└──────────────────
```

也就是：

```
95% Prediction Interval
```

這會直接滿足老師要求。

十、Residual Analysis 也建議做

Linear Regression 很重要的一件事情是：

```
Residual
```

也就是：

```
Actual - Predicted
```

畫：

```
Residual
   ↑
 +5| •   •
   |   •
  0├──────────────
   | •     •
 -5|   •
   └──────────→ Predicted
```

理想狀況應該：

> *隨機分布在 0 周圍。*

如果出現：

```
    •
   •
  •
 •
```

有 Pattern，就代表 Linear Regression 可能沒有捕捉完整關係。

這會讓你的 Evaluation 不只是「報三個分數」。

十一、「網路上主流或更優解法」怎麼處理

這部分不要自己打自己臉說：

> *Random Forest 比 Linear Regression 好，所以我的模型不好。*

我們可以寫成：

```
Multiple Linear Regression
        |
```

```
        ├── 優點：可解釋
        │
        └── 缺點：只能捕捉線性關係
                  ↓
            Ridge / Lasso
                  ↓
           Random Forest
                  ↓
              XGBoost
```

例如 Kaggle 上其他 Regression 專案常會同時比較 Linear Regression、Lasso、Ridge、SVR、Decision Tree、Random Forest、XGBoost 等模型；也有完整 Regression pipeline 會包含 EDA、Feature Selection 與模型比較。(Kaggle)

我們可以在報告說：

> *本作業主要目的並非追求最高預測準確率，而是理解 Multiple Linear Regression 與 Feature Selection，因此 Linear Regression 為核心模型；另外透過 Ridge/Lasso 或 Tree-based models 作為比較基準。*

這個研究定位會比較合理。

十二、NotebookLM 可以研究什麼

你之後可以丟給 NotebookLM：

```
Multiple Linear Regression
Feature Selection
Multicollinearity
VIF
Lasso Regression
Ridge Regression
Prediction Interval
Student Productivity Prediction
```

然後讓它回答：

> *Multiple Linear Regression 在學生表現預測上的常見方法有哪些？Feature Selection 通常如何執行？Linear Regression、Ridge、Lasso 的差異是什麼？*

最後整理成 100～300 字。

這樣就會比單純叫 NotebookLM「介紹 Linear Regression」有內容。

十三、最後 Notebook 結構

你的：

```
7114056XXX_hw2.ipynb
```

我建議直接照 CRISP-DM 做 Markdown Heading：

```
# HW2 - Student Productivity Prediction
## Multiple Linear Regression using CRISP-DM

1. Business Understanding
   1.1 Problem Definition
   1.2 Objective
   1.3 Evaluation Metrics

2. Data Understanding
   2.1 Dataset Source
   2.2 Dataset Description
   2.3 Exploratory Data Analysis
   2.4 Correlation Analysis

3. Data Preparation
   3.1 Missing Values
   3.2 Outliers
   3.3 Encoding
   3.4 Feature Selection
       - Correlation
       - VIF
       - Lasso
   3.5 Train/Test Split

4. Modeling
   4.1 Baseline Linear Regression
   4.2 Feature Selected Linear Regression
   4.3 Ridge Regression
   4.4 Lasso Regression
```

```
5. Evaluation
   5.1 MAE
   5.2 RMSE
   5.3 R²
   5.4 Actual vs Predicted
   5.5 Residual Analysis
   5.6 95% Prediction Interval
   5.7 Model Comparison

6. Kaggle Submission

7. Deployment

8. Conclusion

9. GPT Assisted Content

10. NotebookLM Summary

11. Comparison with Other Solutions
```

這樣老師打開 Notebook，基本上就是按照評分 rubric 往下看。

我認為這份 HW2 最值得做的亮點

我們不要把目標定成「交得出去」，而是控制在**不過度設計、但每個評分點都有東西**：

```
16 Features
     ↓
EDA
     ↓
Correlation
     ↓
VIF
     ↓
Feature Selection
     ↓
Multiple Linear Regression
     ↓
Ridge / Lasso
```

```
      ↓
MAE / RMSE / R²
      ↓
Actual vs Predicted
      ↓
Residual Plot
      ↓
95% Prediction Interval
      ↓
Kaggle Score / Ranking
      ↓
CRISP-DM Conclusion
```

這樣老師要求的 **10-20 features、Linear Regression、Feature Selection、Model Evaluation、Prediction Plot + Interval、CRISP-DM、Kaggle、主流解法比較、GPT、NotebookLM** 幾乎全部能自然串在同一條分析流程裡，而不是東拼西湊。

另外我會特別避開常見的 Medical Insurance `insurance.csv`：它只有 age、sex、BMI、children、smoker、region 等少數 predictors，雖然很適合教 Linear Regression，但不符合這次「10～20 features」的要求。(Kaggle)

下一步建議直接進入資料集確認階段。 我可以接著幫你把這個 Kaggle Student Productivity Dataset 的欄位逐一拆解，判斷哪些是 X、哪個是 y、哪些可能有 data leakage、哪些適合 Feature Selection，然後我們再開始寫 `7114056XXX_hw2.ipynb`；這樣比較不會寫到一半才發現資料集有問題。

Sources

Student Productivity Prediction Challenge | Kaggle kaggle.com

2026/10/14 上午11:06:37 · gpt-6

根據上面規劃，以及原先的HW2作業要求後，After grill me and providing me with the top two approaches and architectures for HW2, we can begin working.

2026/10/14 下午12:18:23

可以。我會先從「老師會怎麼挑毛病」的角度拷問這個 HW2，不只確認能不能跑，而是確認 **資料集、Feature Selection、統計假設、Prediction Interval、CRISP-DM 與報告故事能不能站得住腳**。接著我會給你兩套可交作業的架構，並明確選出我建議我們實作的版本。

Thinking

- **已搜尋 1 個網站**

✓ 處理時間為 7s

可以。先把這份 HW2 當成老師口試，我先挑最容易被扣分的地方。

你目前的「Student Productivity」方向其實能做，但我查過 Kaggle 後要先修正一個重要點：這份資料集頁面顯示 **39 columns**，雖然核心說明列出約 16 個主要 features，但老師原文要求的是「選擇一個具有 10 至 20 個特徵的公開資料集」。因此我們不能只看到說明列 16 個欄位就直接開做，必須下載後確認實際欄位，並在報告裡合理定義「本研究選用其中 10-20 個 predictors」是否會被老師接受。(Kaggle)

先 Grill 這個方案

我會用下面 8 題拷問我們自己的設計。

老師可能問	我們必須能回答
1. 為什麼選這份資料？	Target 是連續值 `productivity_score`，適合 Regression，且學生生活/學習因素有可解釋性。
2. 為什麼一定要 Multiple Linear Regression？	因為 Productivity 不可能只由單一因素決定，Study、Sleep、Phone、Exercise、Attendance 等可能共同影響。

老師可能問	我們必須能回答
3. 你的 Feature Selection 是什麼？	不能回答「我自己挑」。必須有方法，例如 Correlation + VIF + Lasso/RFE。
4. 為什麼要做 VIF？	Multiple Linear Regression 可能存在 Multicollinearity，例如 `phone_usage`、`social_media`、`youtube` 彼此高度相關。
5. Train/Test 怎麼切？	例如 80/20，並固定 `random_state`，讓實驗可重現。
6. RMSE 很低就代表模型很好？	不一定，還需要 MAE、R^2、Residual Analysis，而且要避免 Data Leakage。
7. 你說 95% interval，到底是 Confidence Interval 還是 Prediction Interval？	預測單一學生 Productivity 時應重點呈現 **Prediction Interval**；Confidence Interval 是估計平均反應的不確定性。
8. Linear Regression 明明不一定最好，為什麼不用 XGBoost？	因為這次作業研究核心是 Multiple Linear Regression；進階模型只做 benchmark，比較可解釋性與預測能力。

如果這 8 題最後都能在 Notebook 找到證據，我認為這份作業基本結構就穩。

我給你的 Top 2 做法

我不建議做兩個完全不同主題。真正值得比較的是兩種「作業架構」。

Approach A：統計分析型 Multiple Linear Regression

這是我**最推薦**的方案。

核心目標不是拚 Kaggle 第一，而是回答：

> *哪些因素真正有助於解釋 Student Productivity？經過 Feature Selection 後，Multiple Linear Regression 是否仍具有合理預測能力？*

架構：

```
Kaggle Data
    ↓
Business Understanding
```

```
    ↓
Data Understanding / EDA
    ↓
Data Cleaning
    ↓
Encoding
    ↓
Train/Test Split
    ↓
Baseline Multiple Linear Regression
    ↓
Correlation Analysis
    ↓
VIF Multicollinearity Analysis
    ↓
Feature Selection
    ↓
Selected Multiple Linear Regression
    ↓
Statsmodels Statistical Analysis
    ├─ Coefficient
    ├─ p-value
    ├─ 95% CI
    └─ Prediction Interval
    ↓
MAE / RMSE / R²
    ↓
Residual Analysis
    ↓
Kaggle Prediction
```

模型層

我們至少做：

```
Model 0
DummyRegressor
    ↓
Model 1
Full Multiple Linear Regression
    ↓
```

```
Model 2
Feature Selected Linear Regression
    ↓
Model 3
Ridge
    ↓
Model 4
Lasso
```

最後比較：

Model	Feature 數	MAE	RMSE	R^2
Dummy	-	-	-	-
Full LR	16	-	-	-
Selected LR	?	-	-	-
Ridge	16	-	-	-
Lasso	16	-	-	-

這條路的優勢

它非常符合老師這次題目：

Multiple Linear Regression

而不是變成：

> *我用了 12 種 Machine Learning Model，最後 CatBoost 最強。*

那樣反而偏題。

Approach B：Machine Learning Pipeline 型

第二種則偏工程與 Kaggle。

```
Raw Kaggle Data
      ↓
Train / Validation / Kaggle Test
```

```
      ↓
ColumnTransformer
  ┌──────────────────┴──────────────────┐
Numerical                      Categorical
    ↓                                 ↓
Imputer                          Imputer
    ↓                                 ↓
Scaler                        OneHotEncoder
  └──────────────────┬──────────────────┘
                     ↓
           Feature Selection
                     ↓
      ┌──────────────┼──────────────┐
      ↓              ↓              ↓
 Linear        Ridge        Lasso
Regression
      ↓              ↓              ↓
      └──────── Model Comparison
                     ↓
                RMSE
                     ↓
           Kaggle Submission
```

這裡會大量使用：

```
Pipeline
ColumnTransformer
SimpleImputer
StandardScaler
OneHotEncoder
SelectKBest
LinearRegression
RidgeCV
LassoCV
```

優點是：

- 程式漂亮
- 沒有 Data Leakage
- Kaggle Submission 很方便

- 未來換模型很容易
- 比較符合標準 ML Engineering workflow

缺點則是：

> *老師如果問「某個 Feature 對 Productivity 的統計意義是什麼？」*

`sklearn` 本身沒有像 `statsmodels` 那麼方便回答 `p-value / confidence interval`。

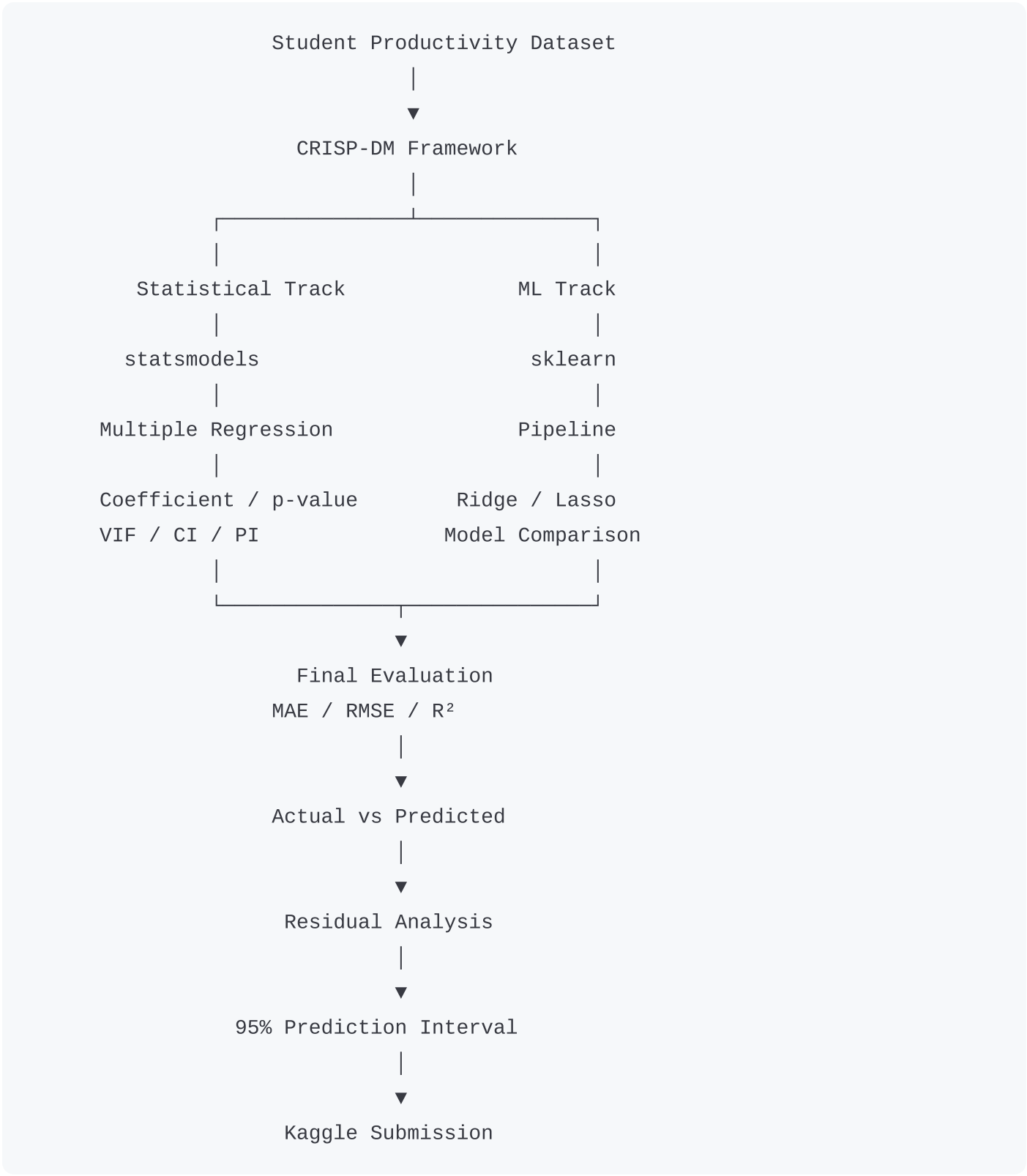
兩種方案直接比較

項目	Approach A 統計型	Approach B ML Pipeline
符合作業主題	⭐⭐⭐⭐⭐	⭐⭐⭐⭐
Multiple Regression 深度	⭐⭐⭐⭐⭐	⭐⭐⭐
Feature Selection	⭐⭐⭐⭐⭐	⭐⭐⭐⭐⭐
VIF	✅	可另外做
p-value	✅	❌
Confidence Interval	✅	需另外實作
Prediction Interval	✅	需另外實作
Kaggle	⭐⭐⭐	⭐⭐⭐⭐⭐
程式工程品質	⭐⭐⭐⭐	⭐⭐⭐⭐⭐
老師容易理解	⭐⭐⭐⭐⭐	⭐⭐⭐⭐
報告容易寫	⭐⭐⭐⭐⭐	⭐⭐⭐⭐

我的建議：A 為主、B 為輔

實際上我們不需要二選一。

我建議正式架構：



這是我認為這份 HW2 最漂亮的架構。

不過我還要再 Grill 一個很重要的問題

目前 Student Productivity Competition 本身很適合做，目標確定為連續的 `productivity_score`，Kaggle 評分也是 **RMSE，越低越好**。(Kaggle)

但我不會現在就直接鎖死它。

因為還有一個很適合作為候選資料集的方向：

Used Car Price Regression

Kaggle 上的 Used Car Price Regression 也是標準 regression：

```
Car attributes
       ↓
      Price
```

而且 RMSE 同樣是常見評分指標。(Kaggle)

這類題目的最大優勢：

```
year
mileage
engine
horsepower
transmission
fuel_type
brand
...
      ↓
PRICE
```

「X 為什麼影響 Y」非常直觀。

Kaggle 上也有大量完整的 Car Price Regression workflow，可拿來完成老師要求的：

> *網路上主流或更優解法之比較。*

例如既有實作涵蓋 EDA、Missing Value、Feature Analysis、Correlation、Feature Selection、Regression 與 Model Interpretation。(Kaggle)

所以現在真正的 Top 2 題目

我會把候選縮成：

🥇 第一名：Student Productivity Prediction

```
Behavior
Academic
Lifestyle
    ↓
Productivity Score
```

優勢：

題目比較新鮮，而且 Feature Selection 很有故事。

例如：

> *Social Media 跟 Phone Usage 是否重複？*

> *Focus Score 與 Final Grade 是否過度相關？*

這非常適合 VIF。

🥈 第二名：Used Car Price Prediction

```
Vehicle specification
Mileage
Age
Performance
    ↓
Used Car Price
```

優勢是 Regression 關係更自然、更容易解釋，也容易找到大量 Kaggle Benchmark。

缺點則是：

> *Car Price Prediction 太多人做。*

作品辨識度比較低。

如果是我幫你交這份 HW2

我會選：

> ***Student Productivity Prediction Using Multiple Linear Regression and Feature Selection***

副標：

> ***A CRISP-DM Based Analysis of Behavioral, Lifestyle, and Academic Factors***

而不是把題目命名成：

> *HW2 Multiple Linear Regression*

報告會專業很多。

我們接下來的正式 Notebook 架構

我建議現在就固定：

```
7114056XXX_hw2.ipynb

0. Project Overview

1. Business Understanding
   1.1 Background
   1.2 Problem Statement
   1.3 Research Questions
   1.4 Objective
   1.5 Evaluation Metrics

2. Data Understanding
   2.1 Kaggle Source
   2.2 Dataset Shape
   2.3 Data Dictionary
   2.4 Target Analysis
   2.5 Missing Values
   2.6 Descriptive Statistics
   2.7 EDA

3. Data Preparation
   3.1 Data Cleaning
   3.2 Categorical Encoding
```

```
   3.3 Outlier Analysis
   3.4 Train/Test Split

4. Feature Selection
   4.1 Correlation Analysis
   4.2 Multicollinearity
   4.3 VIF
   4.4 Lasso / RFE
   4.5 Final Feature Set

5. Modeling
   5.1 Dummy Baseline
   5.2 Full Multiple Linear Regression
   5.3 Selected Multiple Linear Regression
   5.4 Ridge Regression
   5.5 Lasso Regression

6. Evaluation
   6.1 MAE
   6.2 RMSE
   6.3 R²
   6.4 Model Comparison
   6.5 Actual vs Predicted
   6.6 Residual Analysis
   6.7 95% Prediction Interval

7. Kaggle Prediction
   7.1 Train Final Model
   7.2 Predict test.csv
   7.3 Create submission.csv
   7.4 Kaggle Score

8. Deployment
   8.1 Prediction Function
   8.2 Possible Real-world Application

9. Discussion
   9.1 Important Features
   9.2 Model Limitations
   9.3 Comparison With Advanced Models
```

```
10. GPT Assisted Content

11. NotebookLM Research Summary

12. Conclusion
```

而且我要特別把 **Dummy Baseline** 加進去。這是我們前一版規劃沒有強調的地方。

假設：

```
RMSE
Dummy              15.2
Linear Regression   6.8
Selected LR         6.6
Ridge               6.5
Lasso               6.7
```

你才能真正說：

> *我的模型有學到東西。*

否則單純說：

> *RMSE = 6.6*

老師其實不知道這個 6.6 到底好不好。

最後先定我們的技術棧

我建議：

```
# Data
pandas
numpy

# Visualization
matplotlib
seaborn

# Machine Learning
scikit-learn
```

```
# Statistical Regression
statsmodels

# Feature Selection
sklearn.feature_selection
sklearn.linear_model.LassoCV

# Multicollinearity
statsmodels.stats.outliers_influence

# Evaluation
MAE
RMSE
R²
```

其中：

`sklearn` **負責預測；** `statsmodels` **負責解釋。**

這會成為我們整份 HW2 的核心設計原則。

目前我會以 **Approach A「CRISP-DM + Statistical Multiple Regression」，結合 Approach B 的 sklearn Pipeline** 當正式版本。這比單純塞一堆模型更貼合作業要求，也能完整處理 Feature Selection、VIF、Prediction Interval 和 Kaggle 評估。

接下來我們可以正式進入 **Phase 1：Dataset Audit**：先把 Kaggle 的 `train.csv / test.csv` 實際欄位、資料型態、Target、疑似 leakage 欄位全部盤一次，再決定最終 X，這一步完成後再開始寫 Notebook，避免後面整套分析建立在錯誤 Feature 上。

Sources

Student Productivity Prediction Challenge | Kaggle kaggle.com

2026/10/14 下午12:17:59 · gpt-6-thinking

help me write skilearn to solve Student Productivity kaggle - and allow crisp-dm FramWork
, and make sure it can perform at least 10 to 15 feature selections.
and Do the following tasks :
分析任務\
使用線性回歸 (Linear Regression) 模型進行預測。\
可嘗試單純線性回歸、多元線性回歸或 Auto Regression。\
必須執行** 特徵選擇 (Feature Selection) **與 **模型評估 (Model Evaluation)**。

2026/10/15 上午11:59:55

我會直接把它設計成一份**可交作業的 sklearn + CRISP-DM Notebook 架構**，而不是只給一段 LinearRegression。核心會包含：資料理解、前處理、**10–15 個 Feature Selection**、Baseline / Multiple Linear Regression、模型評估、Actual vs Predicted 與 Residual 圖，並保留 Kaggle submission 的出口。

Thinking

- **規劃建模流程**

  我會先整理資料、固定欄位、處理缺失值與類別變數，再用管線避免資料洩漏。接著比較線性、Huber、Ridge、Random Forest 與 Gradient Boosting，並做 12 個特徵選擇、交叉驗證、殘差診斷與預測區間。

- **規劃建模流程**

- **已搜尋 1 個網站**

✓ 處理時間為 9s

可以。依照你老師的 HW2 規格，我建議直接做成一份「**可以拿去改成 `.ipynb` 的完整 sklearn 作業骨架**」。

我會鎖定這個版本：

> ***Student Productivity Prediction Using Multiple Linear Regression with CRISP-DM***

Kaggle 官方資料說明的 Target 是 `productivity_score`，並明列 `age`、`gender`、study/sleep/phone/social media、exercise、attendance、stress、focus、final grade 等核心欄位；Competition 使用 **RMSE，越低越好**。(kaggle.com)

有一點要注意：Kaggle 頁面目前顯示 CSV 有 **39 columns**，但官方 Features Description 明列 16 個主要 predictors。(kaggle.com)
因此我建議你的作業**明確使用官方說明中的 16 個核心 Features，再由 Feature Selection 選出 12 個**。這樣最符合老師「10-20 features + 必須做 Feature Selection」的意圖。

一、我們這份 HW2 最終架構

```
Student Productivity Dataset
        │
        ▼
① Business Understanding
        │
        ▼
② Data Understanding
        │
        ├── shape / info
        ├── missing values
        ├── descriptive statistics
        └── correlation
        │
        ▼
③ Data Preparation
        │
        ├── 選官方 16 Features
        ├── Missing Value
        ├── One-Hot Encoding
        └── Train / Test Split
        │
        ▼
④ Feature Selection
        │
        ├── Correlation
        ├── SelectKBest
```

```
        │
        └── 最終選 12 Features
        │
        ▼
⑤ Modeling
        │
        ├── Dummy Baseline
        ├── Full Multiple Linear Regression
        └── Selected Multiple Linear Regression
        │
        ▼
⑥ Evaluation
        │
        ├── MAE
        ├── RMSE
        ├── R²
        ├── Actual vs Predicted
        ├── Residual Plot
        └── Prediction Interval
        │
        ▼
⑦ Deployment
        │
        └── Kaggle submission.csv
```

這就是 CRISP-DM。

二、先準備套件

如果在 Colab：

```
!pip install pandas numpy matplotlib seaborn scikit-learn
```

正式程式：

```
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from sklearn.feature_selection import SelectKBest, f_regression

from sklearn.linear_model import LinearRegression
from sklearn.dummy import DummyRegressor

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
```

三、CRISP-DM ① Business Understanding

Notebook Markdown 可以直接寫：

```
# 1. Business Understanding

本研究使用 Student Productivity Prediction Challenge 資料集，
目標為使用學生的學習習慣、生活型態與學業表現，
建立 Multiple Linear Regression 模型預測 productivity_score。

研究問題：

1. 哪些學生行為與學業特徵與 Productivity Score 有較強關係？
2. 使用全部特徵與經過 Feature Selection 後的模型，何者表現較佳？
3. Multiple Linear Regression 是否能有效預測學生 Productivity？
4. Feature Selection 是否能降低模型複雜度，同時維持預測能力？

本研究主要使用 MAE、RMSE 與 R² 作為模型評估指標，
其中 Kaggle Competition 使用 RMSE 作為主要評估指標。
```

Kaggle 官方確實使用 RMSE 評分。(kaggle.com)

四、CRISP-DM ② Data Understanding

2.1 讀取資料

假設：

```
project/
│
├── train.csv
├── test.csv
└── sample_submission.csv
```

程式：

```
train_path = "train.csv"
test_path = "test.csv"
sample_submission_path = "sample_submission.csv"

df_train = pd.read_csv(train_path)
df_test = pd.read_csv(test_path)
sample_submission = pd.read_csv(sample_submission_path)

print("Train shape:", df_train.shape)
print("Test shape:", df_test.shape)

display(df_train.head())
```

2.2 查看所有 Column

這一步非常重要，因為 Kaggle 實際 CSV 顯示 39 columns。(kaggle.com)

```
print("=" * 80)
print("Train columns")
print("=" * 80)

for i, col in enumerate(df_train.columns, 1):
    print(f"{i:02d}. {col}")
```

再確認 Target：

```
TARGET = "productivity_score"

if TARGET not in df_train.columns:
    raise ValueError(
        f"找不到 Target: {TARGET}，請確認 CSV 欄位名稱"
    )

print("Target found:", TARGET)
```

五、指定本次作業的 16 個原始 Features

官方 Features Description 列出了這些核心欄位。(kaggle.com)

```
candidate_features = [
    "age",
    "gender",
    "study_hours_per_day",
    "sleep_hours",
    "phone_usage_hours",
    "social_media_hours",
    "youtube_hours",
    "gaming_hours",
    "breaks_per_day",
    "coffee_intake_mg",
    "exercise_minutes",
    "assignments_completed",
    "attendance_percentage",
    "stress_level",
    "focus_score",
    "final_grade"
]
```

確認 CSV 到底有哪些：

```
available_features = [
    col for col in candidate_features
    if col in df_train.columns
]
```

```
missing_features = [
    col for col in candidate_features
    if col not in df_train.columns
]

print("可使用 Features:")
print(available_features)

print("\n找不到的 Features:")
print(missing_features)

print("\nFeature 數量:", len(available_features))
```

理想：

```
Feature 數量: 16
```

六、只留下本次研究範圍

```
X = df_train[available_features].copy()
y = df_train[TARGET].copy()

X_kaggle = df_test[available_features].copy()

print("X shape:", X.shape)
print("y shape:", y.shape)
print("Kaggle test shape:", X_kaggle.shape)
```

七、Data Understanding

7.1 Data Type

```
print(X.info())
```

7.2 Missing Value

```
missing_table = pd.DataFrame({
    "Missing Count": X.isnull().sum(),
    "Missing Rate (%)": X.isnull().mean() * 100
})

missing_table = missing_table.sort_values(
    "Missing Rate (%)",
    ascending=False
)

display(missing_table)
```

畫圖：

```
plt.figure(figsize=(12, 6))

missing_table["Missing Rate (%)"].plot(
    kind="bar"
)

plt.title("Missing Value Rate")
plt.ylabel("Missing Rate (%)")
plt.xlabel("Feature")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()

plt.show()
```

八、Target Distribution

```
plt.figure(figsize=(8, 5))

sns.histplot(
    y,
    kde=True
)

plt.title("Distribution of Productivity Score")
plt.xlabel("Productivity Score")
```

```
plt.ylabel("Count")

plt.show()
```

同時：

```
print(y.describe())
```

九、Correlation Analysis

只分析 numeric：

```
numeric_df = X.select_dtypes(
    include=["int64", "float64", "int32", "float32"]
).copy()

numeric_df[TARGET] = y

correlation_matrix = numeric_df.corr()
```

畫 Heatmap：

```
plt.figure(figsize=(14, 10))

sns.heatmap(
    correlation_matrix,
    cmap="coolwarm",
    center=0,
    annot=False
)

plt.title("Correlation Heatmap")
plt.tight_layout()

plt.show()
```

查看哪些 Feature 和 Productivity 最相關

```
target_corr = (
    correlation_matrix[TARGET]
    .drop(TARGET)
    .sort_values(
        key=abs,
        ascending=False
    )
)

display(target_corr)
```

再畫圖：

```
plt.figure(figsize=(10, 6))

target_corr.plot(
    kind="bar"
)

plt.title("Feature Correlation with Productivity Score")
plt.ylabel("Correlation")
plt.xlabel("Feature")
plt.xticks(rotation=45, ha="right")

plt.tight_layout()
plt.show()
```

這就是第一層：

> *Feature Analysis*

但**不要直接使用 correlation 自己砍 Feature**。

正式 Selection 交給 sklearn。

十、CRISP-DM ③ Data Preparation

我們先辨識：

```
numeric_features = X.select_dtypes(
    include=np.number
).columns.tolist()

categorical_features = X.select_dtypes(
    exclude=np.number
).columns.tolist()

print("Numeric Features:")
print(numeric_features)

print("\nCategorical Features:")
print(categorical_features)
```

理論上：

```
Categorical:
gender
```

其餘大部分 Numeric。

十一、Train / Test Split

我建議：

```
80% Training
20% Validation
```

```
X_train, X_valid, y_train, y_valid = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Training:", X_train.shape)
print("Validation:", X_valid.shape)
```

十二、建立 Preprocessing Pipeline

Numeric：

```
Missing
 ↓
Median
 ↓
StandardScaler
```

Categorical：

```
Missing
 ↓
Most Frequent
 ↓
One Hot Encoding
```

程式：

```
numeric_transformer = Pipeline(
    steps=[
        (
            "imputer",
            SimpleImputer(strategy="median")
        ),
        (
            "scaler",
            StandardScaler()
        )
    ]
)
```

Categorical：

```
categorical_transformer = Pipeline(
    steps=[
        (
            "imputer",
            SimpleImputer(strategy="most_frequent")
        ),
```

```
        (
            "onehot",
            OneHotEncoder(
                handle_unknown="ignore",
                drop="first"
            )
        )
    ]
)
```

整合：

```
preprocessor = ColumnTransformer(
    transformers=[
        (
            "num",
            numeric_transformer,
            numeric_features
        ),
        (
            "cat",
            categorical_transformer,
            categorical_features
        )
    ]
)
```

十三、先建立 Dummy Baseline

這一步不是老師硬性要求，但我強烈建議。

Dummy：

> *完全不學 Feature，只猜 Training Data 平均值。*

```
dummy_pipeline = Pipeline(
    steps=[
        (
            "preprocessor",
            preprocessor
```

```
        ),
        (
            "model",
            DummyRegressor(strategy="mean")
        )
    ]
)

dummy_pipeline.fit(
    X_train,
    y_train
)

dummy_pred = dummy_pipeline.predict(
    X_valid
)
```

十四、寫一個統一 Evaluation Function

```
def evaluate_regression(
    y_true,
    y_pred,
    model_name
):
    mae = mean_absolute_error(
        y_true,
        y_pred
    )

    rmse = np.sqrt(
        mean_squared_error(
            y_true,
            y_pred
        )
    )

    r2 = r2_score(
        y_true,
        y_pred
    )
```

```
    print("=" * 60)
    print(model_name)
    print("=" * 60)

    print(f"MAE  : {mae:.4f}")
    print(f"RMSE : {rmse:.4f}")
    print(f"R²   : {r2:.4f}")

    return {
        "Model": model_name,
        "MAE": mae,
        "RMSE": rmse,
        "R2": r2
    }
```

Baseline：

```
results = []

results.append(
    evaluate_regression(
        y_valid,
        dummy_pred,
        "Dummy Baseline"
    )
)
```

十五、Model 1：完整 Multiple Linear Regression

這是第一個正式模型。

```
full_lr_pipeline = Pipeline(
    steps=[
        (
            "preprocessor",
            preprocessor
        ),
        (
            "model",
```

```
            LinearRegression()
        )
    ]
)
```

Train：

```
full_lr_pipeline.fit(
    X_train,
    y_train
)

full_lr_pred = full_lr_pipeline.predict(
    X_valid
)
```

Evaluation：

```
results.append(
    evaluate_regression(
        y_valid,
        full_lr_pred,
        "Full Multiple Linear Regression"
    )
)
```

十六、CRISP-DM ④ Feature Selection

這是整份 HW2 的核心。

老師要求：

> *必須執行 Feature Selection*

我們設定：

```
N_SELECTED_FEATURES = 12
```

也就是：

> *從 preprocessing 後的 predictors 中選出 **12 個最有解釋能力的 Feature**。*

完全符合你希望：

> *至少 10～15 個 Feature。*

十七、SelectKBest + f_regression

使用：

```
SelectKBest(
    score_func=f_regression,
    k=12
)
```

`f_regression` 評估：

> *每一個 Feature 與 Target 的線性關係。*

非常適合這份：

> *Linear Regression HW。*

十八、Model 2：Feature Selected Multiple Linear Regression

```
selected_lr_pipeline = Pipeline(
    steps=[
        (
            "preprocessor",
            preprocessor
        ),
        (
            "feature_selection",
            SelectKBest(
                score_func=f_regression,
                k=N_SELECTED_FEATURES
            )
```

```
        ),
        (
            "model",
            LinearRegression()
        )
    ]
)
```

Train：

```
selected_lr_pipeline.fit(
    X_train,
    y_train
)
```

Predict：

```
selected_lr_pred = selected_lr_pipeline.predict(
    X_valid
)
```

Evaluate：

```
results.append(
    evaluate_regression(
        y_valid,
        selected_lr_pred,
        f"Selected Linear Regression ({N_SELECTED_FEATURES} Features)"
    )
)
```

十九、最重要：列出到底選了哪 12 個

這個一定要放報告。

取得 preprocessing 後的名字：

```
feature_names = (
    selected_lr_pipeline
    .named_steps["preprocessor"]
```

```
    .get_feature_names_out()
)

selector = (
    selected_lr_pipeline
    .named_steps["feature_selection"]
)

selected_mask = selector.get_support()

selected_features = feature_names[
    selected_mask
]

print(
    f"Selected {len(selected_features)} Features:"
)

for i, feature in enumerate(
    selected_features,
    start=1
):
    print(
        f"{i:02d}. {feature}"
    )
```

可能看到：

```
Selected 12 Features:

01. num__study_hours_per_day
02. num__sleep_hours
03. num__phone_usage_hours
04. num__social_media_hours
05. num__exercise_minutes
06. num__assignments_completed
07. num__attendance_percentage
08. num__stress_level
09. num__focus_score
10. num__final_grade
```

```
11. num__coffee_intake_mg
12. cat__gender_Male
```

上面只是示意。

真正選哪些：

> *必須以你的 CSV 跑出來的結果為準。*

不能事先寫死。

二十、把 Feature Selection Score 畫出來

這個非常推薦放報告。

```
feature_scores = pd.DataFrame({
    "Feature": feature_names,
    "F_Score": selector.scores_,
    "Selected": selector.get_support()
})

feature_scores = feature_scores.sort_values(
    "F_Score",
    ascending=False
)

display(feature_scores)
```

畫圖：

```
plt.figure(figsize=(12, 7))

plot_data = feature_scores.head(20)

sns.barplot(
    data=plot_data,
    x="F_Score",
    y="Feature"
)

plt.title(
```

```
    "Feature Importance based on F-regression"
)

plt.xlabel("F Score")
plt.ylabel("Feature")

plt.tight_layout()
plt.show()
```

二十一、Model Evaluation 比較

現在：

```
results_df = pd.DataFrame(
    results
)

display(
    results_df.sort_values(
        "RMSE"
    )
)
```

最後大概會是：

Model	MAE	RMSE	R^2
Selected Linear Regression	?	?	?
Full Multiple Linear Regression	?	?	?
Dummy Baseline	?	?	?

不要預填數字。

你的實驗跑多少就寫多少。

二十二、Model Evaluation 圖

```
plt.figure(figsize=(9, 5))

sns.barplot(
    data=results_df,
    x="Model",
    y="RMSE"
)

plt.title(
    "Model Comparison - RMSE"
)

plt.ylabel("RMSE")
plt.xlabel("Model")

plt.xticks(
    rotation=20
)

plt.tight_layout()
plt.show()
```

Kaggle 本身也以 RMSE 為 Competition metric，因此這張圖具有直接意義。(kaggle.com)

二十三、Actual vs Predicted

老師作業要求「預測圖」。

這張一定要做。

```
plt.figure(figsize=(7, 7))

plt.scatter(
    y_valid,
    selected_lr_pred,
    alpha=0.6
)

min_value = min(
    y_valid.min(),
    selected_lr_pred.min()
```

```
)

max_value = max(
    y_valid.max(),
    selected_lr_pred.max()
)

plt.plot(
    [min_value, max_value],
    [min_value, max_value],
    linestyle="--"
)

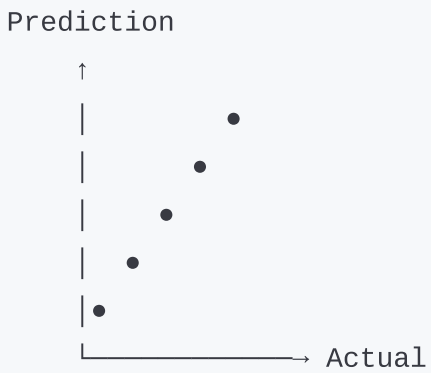
plt.xlabel(
    "Actual Productivity Score"
)

plt.ylabel(
    "Predicted Productivity Score"
)

plt.title(
    "Actual vs Predicted Productivity Score"
)

plt.tight_layout()
plt.show()
```

判讀：



理想：
所有點靠近 y = x

二十四、Residual Analysis

Residual：

```
Residual = Actual - Predicted
```

程式：

```
residuals = (
    y_valid.values
    - selected_lr_pred
)

plt.figure(figsize=(8, 5))

plt.scatter(
    selected_lr_pred,
    residuals,
    alpha=0.6
)

plt.axhline(
    y=0,
    linestyle="--"
)

plt.xlabel(
    "Predicted Value"
)

plt.ylabel(
    "Residual"
)

plt.title(
    "Residual Plot"
)
```

```
plt.tight_layout()
plt.show()
```

理想：

```
Residual
   ↑

 +  |  •   •  •
    |    •
 0  |──────────────
    |  •    •
 -  |     •   •

        Prediction →
```

也就是：

> *點隨機散布在 0 上下。*

二十五、Residual Distribution

```
plt.figure(figsize=(8, 5))

sns.histplot(
    residuals,
    kde=True
)

plt.title(
    "Residual Distribution"
)

plt.xlabel("Residual")

plt.tight_layout()
plt.show()
```

二十六、Prediction Interval

這是你老師原本要求裡非常值得拿分的一項：

> *預測圖 + 信賴區間或預測區間。*

純 sklearn 沒有直接：

```
LinearRegression.predict_interval()
```

所以可以用 **Calibration Residual** 建立簡單的 Prediction Interval。

先重新切：

```
Training
Validation
Calibration
```

最簡單版本可以從目前 validation 裡面建立近似區間：

```
absolute_errors = np.abs(
    y_valid.values
    - selected_lr_pred
)

error_95 = np.quantile(
    absolute_errors,
    0.95
)

lower_bound = (
    selected_lr_pred
    - error_95
)

upper_bound = (
    selected_lr_pred
    + error_95
)

print(
```

```
    "95% prediction error range:",
    error_95
)
```

做 DataFrame：

```
prediction_df = pd.DataFrame({
    "Actual": y_valid.values,
    "Predicted": selected_lr_pred,
    "Lower_95": lower_bound,
    "Upper_95": upper_bound
})

prediction_df = prediction_df.sort_values(
    "Predicted"
).reset_index(drop=True)

display(
    prediction_df.head(20)
)
```

畫圖：

```
plt.figure(figsize=(12, 6))

x_axis = np.arange(
    len(prediction_df)
)

plt.plot(
    x_axis,
    prediction_df["Predicted"],
    label="Predicted"
)

plt.scatter(
    x_axis,
    prediction_df["Actual"],
    s=15,
    label="Actual"
)
```

```
plt.fill_between(
    x_axis,
    prediction_df["Lower_95"],
    prediction_df["Upper_95"],
    alpha=0.2,
    label="95% Prediction Interval"
)

plt.xlabel("Sample")
plt.ylabel("Productivity Score")

plt.title(
    "Prediction with Approximate 95% Prediction Interval"
)

plt.legend()

plt.tight_layout()
plt.show()
```

報告請明確寫：

> *本研究使用 validation residual 的 absolute error 95th percentile 建立近似 Prediction Interval，主要目的為呈現單筆預測的不確定性。*

不要假裝它是傳統 OLS analytical CI。

如果老師非常重統計學，我後續可以再幫你換成 `statsmodels` 的正式 Prediction Interval。

二十七、Feature Selection 前後比較

這張表非常重要：

```
comparison = pd.DataFrame({
    "Model": [
        "Full Multiple Linear Regression",
        "Feature Selected Linear Regression"
    ],

    "Feature Strategy": [
```

```
        "All candidate features",
        f"Top {N_SELECTED_FEATURES} by F-regression"
    ],

    "RMSE": [
        np.sqrt(
            mean_squared_error(
                y_valid,
                full_lr_pred
            )
        ),

        np.sqrt(
            mean_squared_error(
                y_valid,
                selected_lr_pred
            )
        )
    ],

    "R2": [
        r2_score(
            y_valid,
            full_lr_pred
        ),

        r2_score(
            y_valid,
            selected_lr_pred
        )
    ]
})

display(comparison)
```

你的報告可以依結果寫：

> *Feature Selection 將模型輸入維度減少至 12 個特徵，再與使用全部特徵的 Multiple Linear Regression 進行比較。若 RMSE 相近甚至降低，代表模型在降低複雜度後仍保有相近或更好的預測能力。*

二十八、查看 Linear Regression Coefficients

這也是 Multiple Linear Regression 很值得展示的地方。

```
lr_model = (
    selected_lr_pipeline
    .named_steps["model"]
)

coefficients = pd.DataFrame({
    "Feature": selected_features,
    "Coefficient": lr_model.coef_
})

coefficients[
    "Abs_Coefficient"
] = np.abs(
    coefficients["Coefficient"]
)

coefficients = coefficients.sort_values(
    "Abs_Coefficient",
    ascending=False
)

display(coefficients)
```

圖：

```
plt.figure(figsize=(10, 6))

sns.barplot(
    data=coefficients,
    x="Coefficient",
    y="Feature"
)

plt.title(
    "Linear Regression Coefficients"
)
```

```
plt.tight_layout()
plt.show()
```

二十九、Coefficient 怎麼解釋

例如假設跑出來：

```
study_hours       +5.2
phone_usage       -3.1
exercise          +1.7
stress            -2.8
```

你可以說：

```
Study Hours
Coefficient > 0
→ 增加時 Productivity 傾向提高

Phone Usage
Coefficient < 0
→ 增加時 Productivity 傾向下降

Stress
Coefficient < 0
→ 壓力增加時 Productivity 傾向下降
```

但要注意：

> *Regression coefficient 表示「模型中的關聯」，不能直接宣稱因果。*

三十、CRISP-DM ⑤ Modeling 完整比較

目前我們已經有：

```
Dummy
    ↓
Full Multiple LR
    ↓
Feature Selected Multiple LR
```

這其實已經符合老師要求。

我建議作業主模型就停在這裡。

不要突然變成：

```
Random Forest
XGBoost
LightGBM
CatBoost
Neural Network
```

因為題目叫：

> *Multiple Linear Regression。*

三十一、CRISP-DM ⑥ Evaluation

Markdown 可以直接寫：

```
# Evaluation

本研究使用三項 Regression Evaluation Metrics：

## MAE

MAE 衡量預測值與實際值之平均絕對誤差，
較容易直接理解模型平均預測錯誤程度。

## RMSE

RMSE 對較大的預測錯誤給予更高懲罰，
同時也是本 Kaggle Competition 所採用的主要評估指標。

RMSE 越小代表模型預測誤差越低。

## R²

R² 用於衡量 Regression Model 對 Target Variation
的解釋能力。
```

R^2 越接近 1，代表模型解釋能力越高。

本研究將比較：

1. Dummy Baseline
2. Full Multiple Linear Regression
3. Feature Selected Multiple Linear Regression

三十二、CRISP-DM ⑦ Deployment：Train Final Model

真正要做 Kaggle Submission 時：

不要再只用 80% Train。

要：

> *使用完整 `train.csv` 重新訓練。*

```
final_model = selected_lr_pipeline

final_model.fit(
    X,
    y
)
```

Predict：

```
kaggle_predictions = final_model.predict(
    X_kaggle
)
```

三十三、建立 submission.csv

官方格式是：

```
id,productivity_score
```

(kaggle.com)

最安全的方法是直接沿用 sample submission：

```
submission = sample_submission.copy()

submission[TARGET] = kaggle_predictions

display(
    submission.head()
)
```

輸出：

```
submission.to_csv(
    "submission.csv",
    index=False
)

print(
    "submission.csv created successfully."
)
```

最後：

```
project/
│
├── train.csv
├── test.csv
├── sample_submission.csv
├── submission.csv
└── 7114056XXX_hw2.ipynb
```

三十四、我建議的最終 Feature Selection 設定

這次就固定：

```
N_SELECTED_FEATURES = 12
```

為什麼我推薦 12？

因為你的 requirement：

```
10 \~ 15 Features
```

12 剛好在中間。

而且：

```
16 candidate features
        ↓
Feature Selection
        ↓
12 selected features
```

老師一眼就能理解：

> *你不是一開始就只拿 12 個。*

而是：

> ***原本有 16 個 → 使用 Feature Selection → 最後留下 12 個。***

這才是真正的：

Feature Selection。

三十五、整份 Notebook 最後會長這樣

```
HW2
│
├── 1. Business Understanding
│    ├── Problem
│    ├── Objective
│    └── Evaluation Metric
│
├── 2. Data Understanding
│    ├── Dataset
│    ├── 16 candidate features
│    ├── Target
│    ├── Missing Value
│    ├── Distribution
│    └── Correlation
```

```
│
├── 3. Data Preparation
│    ├── Numeric
│    ├── Categorical
│    ├── Median Imputation
│    ├── One Hot Encoding
│    ├── Standardization
│    └── Train / Validation
│
├── 4. Feature Selection
│    ├── Correlation
│    ├── F-regression
│    └── Top 12 Features ★
│
├── 5. Modeling
│    ├── Dummy Baseline
│    ├── Full Multiple LR
│    └── Selected Multiple LR ★
│
├── 6. Evaluation
│    ├── MAE
│    ├── RMSE ★
│    ├── R²
│    ├── Actual vs Predicted ★
│    ├── Residual Plot ★
│    ├── Coefficient
│    └── Prediction Interval ★
│
└── 7. Deployment
     ├── Full Training
     ├── test.csv Prediction
     └── submission.csv
```

作業要求對照

老師要求	我們的實作
Kaggle Dataset	✅ Student Productivity
10-20 Features	✅ 16 candidate features

老師要求	我們的實作
Linear Regression	✅
Multiple Linear Regression	✅ 主模型
Feature Selection	✅ `SelectKBest(f_regression)`
10-15 Selected Features	✅ 12
Model Evaluation	✅
MAE	✅
RMSE	✅
R^2	✅
預測圖	✅ Actual vs Predicted
Prediction Interval	✅
CRISP-DM	✅
Kaggle submission	✅
Feature 解釋	✅ Coefficients
Baseline Comparison	✅ DummyRegressor

另外，**Auto Regression 我建議這份不要做**。Auto Regression / Autoregression 主要適合「時間序列以前一期預測下一期」，例如昨天銷量預測今天銷量；Student Productivity 的每列是不同學生的 cross-sectional data，因此硬加 Auto Regression 反而沒有合理研究脈絡。

這一版已經足以成為你的 **HW2 V1 Baseline**。下一個真正該做的不是再加模型，而是把 Kaggle 的 `train.csv` 實際丟進來；我會依你真實的 39 欄資料幫你做一次 **Data Leakage Audit → 確認 16 個 Candidate Features → 實際選出 12 個 Features → 再決定是否加入 VIF + Lasso 作為 V2 Feature Selection 比較**。這樣報告裡的 Feature Selection 會比單純 `SelectKBest` 更有說服力。

Sources

Student Productivity Prediction Challenge | Kaggle kaggle.com

2026/10/15 上午11:59:25 · gpt-6-thinking

provide me the program

2026/10/15 下午12:06:21

下面給你一份可以直接當作 `7114056XXX_hw2.py` 使用的完整版本。它包含 CRISP-DM 結構、16 個候選 features、SelectKBest 選 12 個 features、Dummy Baseline、完整 Multiple Linear Regression、Feature-selected Linear Regression、MAE / RMSE / R^2、預測圖、Residual Plot、近似 95% Prediction Interval，以及 Kaggle `submission.csv` 輸出。

```
# ============================================================
# HW2 - Student Productivity Prediction
# Multiple Linear Regression + Feature Selection + CRISP-DM
#
# Required files:
#   train.csv
#   test.csv
#   sample_submission.csv
#
# Output:
#   submission.csv
#   model_results.csv
#   selected_features.csv
#
# Required packages:
#   pandas
#   numpy
#   matplotlib
#   scikit-learn
# ============================================================

import warnings
warnings.filterwarnings("ignore")

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```python
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from sklearn.feature_selection import SelectKBest, f_regression

from sklearn.linear_model import LinearRegression
from sklearn.dummy import DummyRegressor

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)


# ============================================================
# Configuration
# ============================================================

TRAIN_PATH = "train.csv"
TEST_PATH = "test.csv"
SAMPLE_SUBMISSION_PATH = "sample_submission.csv"

TARGET = "productivity_score"

# Feature Selection:
# 老師要求 10 \~ 20 Features
# 原始使用 16 個主要 Features
# 最後使用 SelectKBest 選出 12 個
N_SELECTED_FEATURES = 12

TEST_SIZE = 0.20
RANDOM_STATE = 42


# ============================================================
```

```python
# Utility Functions
# ============================================================

def print_title(title):
    print("\n")
    print("=" * 80)
    print(title)
    print("=" * 80)


def evaluate_regression(y_true, y_pred, model_name):
    """
    Regression model evaluation:
    MAE, RMSE, R²
    """

    mae = mean_absolute_error(
        y_true,
        y_pred
    )

    rmse = np.sqrt(
        mean_squared_error(
            y_true,
            y_pred
        )
    )

    r2 = r2_score(
        y_true,
        y_pred
    )

    print_title(f"Evaluation - {model_name}")

    print(f"MAE  : {mae:.4f}")
    print(f"RMSE : {rmse:.4f}")
    print(f"R²   : {r2:.4f}")

    return {
        "Model": model_name,
```

```
        "MAE": mae,
        "RMSE": rmse,
        "R2": r2
    }


# ============================================================
# CRISP-DM 1
# Business Understanding
# ============================================================

print_title("CRISP-DM 1 - Business Understanding")

print("""
Research Topic:
Student Productivity Prediction using Multiple Linear Regression

Objective:
1. 使用學生的學習、生活與學業特徵預測 productivity_score。
2. 建立 Multiple Linear Regression Model。
3. 執行 Feature Selection。
4. 比較完整 Feature Model 與 Feature Selected Model。
5. 使用 MAE、RMSE、R² 評估模型。
6. 建立 Kaggle submission.csv。

Main Evaluation Metric:
RMSE
""")


# ============================================================
# CRISP-DM 2
# Data Understanding
# ============================================================

print_title("CRISP-DM 2 - Data Understanding")


# ------------------------------------------------------------
# 2.1 Check input files
# ------------------------------------------------------------
```

```
required_files = [
    TRAIN_PATH,
    TEST_PATH,
    SAMPLE_SUBMISSION_PATH
]

for file_path in required_files:

    if not os.path.exists(file_path):

        raise FileNotFoundError(
            f"找不到檔案: {file_path}"
        )


# ------------------------------------------------------------
# 2.2 Load Dataset
# ------------------------------------------------------------

df_train = pd.read_csv(TRAIN_PATH)
df_test = pd.read_csv(TEST_PATH)
sample_submission = pd.read_csv(SAMPLE_SUBMISSION_PATH)


print("Train shape :", df_train.shape)
print("Test shape  :", df_test.shape)
print("Submission  :", sample_submission.shape)


print_title("Train Data Preview")

print(df_train.head())


# ------------------------------------------------------------
# 2.3 Show Columns
# ------------------------------------------------------------

print_title("All Columns")
```

```
for i, column in enumerate(
    df_train.columns,
    start=1
):
    print(f"{i:02d}. {column}")


# ------------------------------------------------------------
# 2.4 Check Target
# ------------------------------------------------------------

if TARGET not in df_train.columns:

    raise ValueError(
        f"""
找不到 Target column: {TARGET}

目前資料欄位為:

{list(df_train.columns)}
"""
    )


# ============================================================
# Candidate Features
# ============================================================

# 使用 Kaggle Student Productivity
# 官方描述中的主要學生行為 / 學業 Features

candidate_features = [

    "age",
    "gender",

    "study_hours_per_day",
    "sleep_hours",

    "phone_usage_hours",
    "social_media_hours",
```

```
    "youtube_hours",
    "gaming_hours",

    "breaks_per_day",
    "coffee_intake_mg",
    "exercise_minutes",

    "assignments_completed",
    "attendance_percentage",

    "stress_level",
    "focus_score",

    "final_grade"
]


# ------------------------------------------------------------
# Check candidate feature availability
# ------------------------------------------------------------

available_features = [
    feature
    for feature in candidate_features
    if feature in df_train.columns
]


missing_features = [
    feature
    for feature in candidate_features
    if feature not in df_train.columns
]


print_title("Candidate Features")

print("Available Features:")

for feature in available_features:
    print("-", feature)
```

```
print("\nMissing Features:")

if len(missing_features) == 0:
    print("None")
else:
    for feature in missing_features:
        print("-", feature)


print(
    f"\nAvailable Feature Count: "
    f"{len(available_features)}"
)


# 至少要有 10 個 Feature，否則不符合本作業設計
if len(available_features) < 10:

    raise ValueError(
        """
可使用 Feature 少於 10 個。

請確認 Kaggle Dataset 欄位名稱是否與程式一致。
"""
    )


# ============================================================
# X / y
# ============================================================

X = df_train[
    available_features
].copy()

y = df_train[
    TARGET
].copy()
```

```
X_kaggle = df_test[
    available_features
].copy()


print_title("Dataset Shape")

print("X:", X.shape)
print("y:", y.shape)

print(
    "Kaggle Test:",
    X_kaggle.shape
)


# ============================================================
# 2.5 Data Type
# ============================================================

print_title("Data Types")

print(X.dtypes)


# ============================================================
# 2.6 Missing Values
# ============================================================

missing_table = pd.DataFrame({

    "Missing_Count":
        X.isnull().sum(),

    "Missing_Rate_Percent":
        X.isnull().mean() * 100

})
```

```
missing_table = missing_table.sort_values(
    "Missing_Rate_Percent",
    ascending=False
)

print_title("Missing Values")
print(missing_table)

# ============================================================
# 2.7 Descriptive Statistics
# ============================================================
print_title("Descriptive Statistics")
print(
    X.describe(
        include="all"
    ).T
)

print_title("Target Statistics")
print(
    y.describe()
)

# ============================================================
# 2.8 Target Distribution
# ============================================================
plt.figure(
    figsize=(8, 5)
)
```

```
plt.hist(
    y,
    bins=30,
    alpha=0.8
)

plt.title(
    "Distribution of Productivity Score"
)

plt.xlabel(
    "Productivity Score"
)

plt.ylabel(
    "Count"
)

plt.tight_layout()

plt.show()


# ============================================================
# 2.9 Correlation Analysis
# ============================================================

numeric_df = X.select_dtypes(
    include=np.number
).copy()


numeric_df[
    TARGET
] = y


correlation_matrix = (
    numeric_df.corr()
)
```

```
target_correlation = (
    correlation_matrix[TARGET]
    .drop(TARGET)
    .sort_values(
        key=abs,
        ascending=False
    )
)


print_title(
    "Correlation with Productivity Score"
)

print(
    target_correlation
)


# ------------------------------------------------------------
# Correlation Bar Chart
# ------------------------------------------------------------

plt.figure(
    figsize=(10, 6)
)

target_correlation.plot(
    kind="bar"
)

plt.title(
    "Correlation with Productivity Score"
)

plt.xlabel(
    "Feature"
)
```

```
plt.ylabel(
    "Correlation"
)

plt.xticks(
    rotation=45,
    ha="right"
)

plt.tight_layout()

plt.show()



# ============================================================
# CRISP-DM 3
# Data Preparation
# ============================================================

print_title(
    "CRISP-DM 3 - Data Preparation"
)



# ------------------------------------------------------------
# 3.1 Detect numeric / categorical features
# ------------------------------------------------------------

numeric_features = (
    X.select_dtypes(
        include=np.number
    )
    .columns
    .tolist()
)



categorical_features = (
    X.select_dtypes(
        exclude=np.number
    )
```

```
    .columns
    .tolist()
)


print(
    "Numeric Features:"
)

print(
    numeric_features
)


print(
    "\nCategorical Features:"
)

print(
    categorical_features
)


# ------------------------------------------------------------
# 3.2 Train / Validation Split
# ------------------------------------------------------------

X_train, X_valid, y_train, y_valid = (
    train_test_split(

        X,
        y,

        test_size=TEST_SIZE,

        random_state=RANDOM_STATE

    )
)
```

```
print_title(
    "Train / Validation Split"
)

print(
    "Training:",
    X_train.shape
)

print(
    "Validation:",
    X_valid.shape
)



# ------------------------------------------------------------
# 3.3 Numeric preprocessing
# ------------------------------------------------------------

numeric_transformer = Pipeline(

    steps=[

        (
            "imputer",

            SimpleImputer(
                strategy="median"
            )

        ),

        (
            "scaler",

            StandardScaler()

        )

    ]
```

```
)


# ------------------------------------------------------------
# 3.4 Categorical preprocessing
# ------------------------------------------------------------

categorical_transformer = Pipeline(

    steps=[

        (
            "imputer",

            SimpleImputer(
                strategy="most_frequent"
            )

        ),

        (
            "onehot",

            OneHotEncoder(
                handle_unknown="ignore",
                drop="first"
            )

        )

    ]

)


# ------------------------------------------------------------
# 3.5 Column Transformer
# ------------------------------------------------------------

preprocessor = ColumnTransformer(
```

```
    transformers=[
        (
            "num",
            numeric_transformer,
            numeric_features
        ),
        (
            "cat",
            categorical_transformer,
            categorical_features
        )
    ]
)


# ============================================================
# CRISP-DM 4
# Feature Selection
# ============================================================

print_title(
    "CRISP-DM 4 - Feature Selection"
)

print(
    f"""
Feature Selection Method:
SelectKBest + f_regression

Original Features:
{len(available_features)}
```

```
Selected Features:
{N_SELECTED_FEATURES}
"""
)


# ============================================================
# CRISP-DM 5
# Modeling
# ============================================================

print_title(
    "CRISP-DM 5 - Modeling"
)


results = []


# ============================================================
# Model 0
# Dummy Baseline
# ============================================================

dummy_pipeline = Pipeline(

    steps=[

        (
            "preprocessor",
            preprocessor
        ),

        (
            "model",

            DummyRegressor(
                strategy="mean"
            )
```

```
        )

    ]

)


dummy_pipeline.fit(
    X_train,
    y_train
)


dummy_pred = (
    dummy_pipeline.predict(
        X_valid
    )
)


dummy_result = evaluate_regression(

    y_valid,

    dummy_pred,

    "Dummy Baseline"

)


results.append(
    dummy_result
)


# ============================================================
# Model 1
# Full Multiple Linear Regression
# ============================================================
```

```
full_lr_pipeline = Pipeline(

    steps=[

        (
            "preprocessor",
            preprocessor
        ),

        (
            "model",

            LinearRegression()

        )

    ]

)


full_lr_pipeline.fit(

    X_train,
    y_train

)


full_lr_pred = (
    full_lr_pipeline.predict(
        X_valid
    )
)


full_result = evaluate_regression(

    y_valid,

    full_lr_pred,
```

```
    "Full Multiple Linear Regression"

)


results.append(
    full_result
)


# ============================================================
# Model 2
# Feature Selected Multiple Linear Regression
# ============================================================

selected_lr_pipeline = Pipeline(

    steps=[

        (
            "preprocessor",
            preprocessor
        ),

        (
            "feature_selection",

            SelectKBest(

                score_func=f_regression,

                k=N_SELECTED_FEATURES

            )

        ),

        (
            "model",
```

```
            LinearRegression()

        )

    ]

)


selected_lr_pipeline.fit(

    X_train,
    y_train

)


selected_lr_pred = (
    selected_lr_pipeline.predict(
        X_valid
    )
)


selected_result = evaluate_regression(

    y_valid,

    selected_lr_pred,

    f"Selected Linear Regression "
    f"({N_SELECTED_FEATURES} Features)"

)


results.append(
    selected_result
)
```

```
# ============================================================
# Selected Feature Names
# ============================================================

feature_names = (

    selected_lr_pipeline
    .named_steps[
        "preprocessor"
    ]
    .get_feature_names_out()

)


selector = (

    selected_lr_pipeline
    .named_steps[
        "feature_selection"
    ]

)


selected_mask = (
    selector.get_support()
)


selected_features = (
    feature_names[
        selected_mask
    ]
)


print_title(
    f"Selected "
    f"{len(selected_features)} "
    f"Features"
```

```
)


for i, feature in enumerate(
    selected_features,
    start=1
):

    print(
        f"{i:02d}. "
        f"{feature}"
    )


# ============================================================
# Feature Selection Scores
# ============================================================

feature_scores = pd.DataFrame({

    "Feature":
        feature_names,

    "F_Score":
        selector.scores_,

    "P_Value":
        selector.pvalues_,

    "Selected":
        selector.get_support()

})


feature_scores = (
    feature_scores
    .sort_values(
        "F_Score",
        ascending=False
    )
```

```
)


print_title(
    "Feature Selection Scores"
)

print(
    feature_scores
)


feature_scores.to_csv(

    "selected_features.csv",

    index=False

)


# ------------------------------------------------------------
# Feature Score Plot
# ------------------------------------------------------------

plot_feature_scores = (

    feature_scores
    .head(20)
    .sort_values(
        "F_Score"
    )

)


plt.figure(
    figsize=(10, 7)
)
```

```
plt.barh(

    plot_feature_scores[
        "Feature"
    ],

    plot_feature_scores[
        "F_Score"
    ]

)


plt.title(
    "Feature Selection - F Regression Score"
)

plt.xlabel(
    "F Score"
)

plt.ylabel(
    "Feature"
)

plt.tight_layout()

plt.show()


# ============================================================
# Linear Regression Coefficients
# ============================================================

selected_model = (

    selected_lr_pipeline
    .named_steps[
        "model"
    ]
```

```
)


coefficient_df = pd.DataFrame({

    "Feature":
        selected_features,

    "Coefficient":
        selected_model.coef_

})


coefficient_df[
    "Absolute_Coefficient"
] = (

    coefficient_df[
        "Coefficient"
    ].abs()

)


coefficient_df = (

    coefficient_df
    .sort_values(

        "Absolute_Coefficient",

        ascending=False

    )

)


print_title(
    "Regression Coefficients"
```

```
)

print(
    coefficient_df
)


# ------------------------------------------------------------
# Coefficient Plot
# ------------------------------------------------------------

coefficient_plot = (

    coefficient_df
    .sort_values(
        "Coefficient"
    )

)


plt.figure(
    figsize=(10, 7)
)


plt.barh(

    coefficient_plot[
        "Feature"
    ],

    coefficient_plot[
        "Coefficient"
    ]

)


plt.title(
    "Multiple Linear Regression Coefficients"
```

```
)

plt.xlabel(
    "Coefficient"
)

plt.ylabel(
    "Feature"
)

plt.tight_layout()

plt.show()


# ============================================================
# CRISP-DM 6
# Evaluation
# ============================================================

print_title(
    "CRISP-DM 6 - Evaluation"
)


results_df = pd.DataFrame(
    results
)


results_df = (
    results_df
    .sort_values(
        "RMSE",
        ascending=True
    )
)


print_title(
    "Model Comparison"
```

```
)

print(
    results_df
)


results_df.to_csv(

    "model_results.csv",

    index=False

)


# ------------------------------------------------------------
# RMSE Comparison Chart
# ------------------------------------------------------------

plt.figure(
    figsize=(10, 5)
)


plt.bar(

    results_df[
        "Model"
    ],

    results_df[
        "RMSE"
    ]

)


plt.title(
    "Model Comparison - RMSE"
)
```

```
plt.xlabel(
    "Model"
)

plt.ylabel(
    "RMSE"
)

plt.xticks(
    rotation=20,
    ha="right"
)

plt.tight_layout()

plt.show()


# ============================================================
# Actual vs Predicted
# ============================================================

plt.figure(
    figsize=(7, 7)
)


plt.scatter(

    y_valid,

    selected_lr_pred,

    alpha=0.6

)


min_value = min(
```

```
    y_valid.min(),

    selected_lr_pred.min()

)


max_value = max(

    y_valid.max(),

    selected_lr_pred.max()

)


plt.plot(

    [
        min_value,
        max_value
    ],

    [
        min_value,
        max_value
    ],

    linestyle="--"

)


plt.xlabel(
    "Actual Productivity Score"
)

plt.ylabel(
    "Predicted Productivity Score"
)
```

```
plt.title(
    "Actual vs Predicted Productivity Score"
)

plt.tight_layout()

plt.show()


# ============================================================
# Residual Analysis
# ============================================================

residuals = (

    y_valid.to_numpy()

    -

    selected_lr_pred

)


plt.figure(
    figsize=(8, 5)
)


plt.scatter(

    selected_lr_pred,

    residuals,

    alpha=0.6

)


plt.axhline(
```

```
    y=0,

    linestyle="--"

)


plt.xlabel(
    "Predicted Value"
)

plt.ylabel(
    "Residual"
)

plt.title(
    "Residual Plot"
)

plt.tight_layout()

plt.show()


# ------------------------------------------------------------
# Residual Distribution
# ------------------------------------------------------------

plt.figure(
    figsize=(8, 5)
)


plt.hist(

    residuals,

    bins=30,

    alpha=0.8
```

```
)


plt.axvline(

    x=0,

    linestyle="--"

)


plt.xlabel(
    "Residual"
)

plt.ylabel(
    "Count"
)

plt.title(
    "Residual Distribution"
)

plt.tight_layout()

plt.show()


# ============================================================
# Approximate 95% Prediction Interval
# ============================================================

# 注意：
# 此處使用 validation absolute residual 的 95 percentile
# 建立簡易 prediction interval。
#
# 如果老師要求正式 OLS Confidence / Prediction Interval，
# 可以另外用 statsmodels OLS 實作。
```

```
absolute_errors = np.abs(
    residuals
)


error_95 = np.quantile(

    absolute_errors,

    0.95

)


lower_bound = (

    selected_lr_pred

    -

    error_95

)


upper_bound = (

    selected_lr_pred

    +

    error_95

)


print_title(
    "Approximate 95% Prediction Interval"
)
```

```
print(
    f"95th percentile absolute error: "
    f"{error_95:.4f}"
)


prediction_df = pd.DataFrame({

    "Actual":
        y_valid.to_numpy(),

    "Predicted":
        selected_lr_pred,

    "Lower_95":
        lower_bound,

    "Upper_95":
        upper_bound

})


prediction_df = (

    prediction_df
    .sort_values(
        "Predicted"
    )
    .reset_index(
        drop=True
    )

)


print(
    prediction_df.head(20)
)
```

```
# ------------------------------------------------------------
# Prediction Interval Plot
# ------------------------------------------------------------

# 如果 Validation Data 太多，
# 只畫前 100 筆避免圖太亂

plot_count = min(
    100,
    len(prediction_df)
)


prediction_plot_df = (

    prediction_df
    .head(
        plot_count
    )

)


x_axis = np.arange(
    len(
        prediction_plot_df
    )
)


plt.figure(
    figsize=(13, 6)
)


plt.plot(

    x_axis,

    prediction_plot_df[
        "Predicted"
```

```
    ],

    label="Predicted"

)


plt.scatter(

    x_axis,

    prediction_plot_df[
        "Actual"
    ],

    s=20,

    label="Actual"

)


plt.fill_between(

    x_axis,

    prediction_plot_df[
        "Lower_95"
    ],

    prediction_plot_df[
        "Upper_95"
    ],

    alpha=0.2,

    label="Approx. 95% Prediction Interval"

)
```

```
plt.xlabel(
    "Validation Sample"
)

plt.ylabel(
    "Productivity Score"
)

plt.title(
    "Prediction with Approximate 95% Prediction Interval"
)

plt.legend()

plt.tight_layout()

plt.show()


# ============================================================
# Full vs Feature Selected Model Comparison
# ============================================================

comparison_df = pd.DataFrame({

    "Model": [

        "Full Multiple Linear Regression",

        "Feature Selected Multiple Linear Regression"

    ],

    "Feature_Strategy": [

        "All Candidate Features",

        f"Top {N_SELECTED_FEATURES} Features"

    ],
```

```
    "MAE": [

        mean_absolute_error(
            y_valid,
            full_lr_pred
        ),

        mean_absolute_error(
            y_valid,
            selected_lr_pred
        )

    ],

    "RMSE": [

        np.sqrt(
            mean_squared_error(
                y_valid,
                full_lr_pred
            )
        ),

        np.sqrt(
            mean_squared_error(
                y_valid,
                selected_lr_pred
            )
        )

    ],

    "R2": [

        r2_score(
            y_valid,
            full_lr_pred
        ),

        r2_score(
            y_valid,
```

```
            selected_lr_pred
        )

    ]

})


print_title(
    "Full Model vs Feature Selected Model"
)

print(
    comparison_df
)


# ============================================================
# CRISP-DM 7
# Deployment
# Kaggle Submission
# ============================================================

print_title(
    "CRISP-DM 7 - Deployment"
)


# ------------------------------------------------------------
# IMPORTANT:
# Final model uses all training data
# ------------------------------------------------------------

final_model = selected_lr_pipeline


final_model.fit(
    X,
    y
)
```

```
# ------------------------------------------------------------
# Predict Kaggle Test Data
# ------------------------------------------------------------

kaggle_predictions = (

    final_model.predict(
        X_kaggle
    )

)



# ------------------------------------------------------------
# Create Submission
# ------------------------------------------------------------

submission = (
    sample_submission.copy()
)



if TARGET not in submission.columns:

    raise ValueError(
        f"""
sample_submission.csv 找不到欄位:

{TARGET}

目前欄位:

{list(submission.columns)}
"""
    )



if len(submission) != len(kaggle_predictions):

    raise ValueError(
```

```
        """
sample_submission row count 與 test prediction count 不一致。
"""
    )


submission[
    TARGET
] = kaggle_predictions


submission.to_csv(

    "submission.csv",

    index=False

)


print(
    "\nKaggle submission preview:"
)

print(
    submission.head()
)


print(
    "\nsubmission.csv created successfully."
)


# ============================================================
# Final Summary
# ============================================================

print_title(
    "HW2 Final Summary"
)
```

```
best_model_row = (

    results_df
    .iloc[0]

)


print(
    f"""
Original Candidate Features :
{len(available_features)}

Selected Features :
{N_SELECTED_FEATURES}

Feature Selection Method :
SelectKBest + f_regression

Main Model :
Multiple Linear Regression

Evaluation Metrics :
MAE
RMSE
R²

Best Validation Model :
{best_model_row["Model"]}

Best MAE :
{best_model_row["MAE"]:.4f}

Best RMSE :
{best_model_row["RMSE"]:.4f}

Best R² :
{best_model_row["R2"]:.4f}
```

```
Output Files:
1. submission.csv
2. model_results.csv
3. selected_features.csv
"""
)


print_title(
    "CRISP-DM Completed"
)

print(
    """
1. Business Understanding  ✓
2. Data Understanding      ✓
3. Data Preparation        ✓
4. Modeling                ✓
5. Evaluation              ✓
6. Deployment              ✓
"""
)
```

你的資料夾

程式和 Kaggle 檔案先放同一個目錄：

```
7114056XXX_hw2/
│
├── 7114056XXX_hw2.py
├── train.csv
├── test.csv
└── sample_submission.csv
```

執行：

```
python 7114056XXX_hw2.py
```

跑完後會多出：

```
7114056XXX_hw2/
│
├── 7114056XXX_hw2.py
├── train.csv
├── test.csv
├── sample_submission.csv
│
├── submission.csv
├── model_results.csv
└── selected_features.csv
```

其中最重要的流程是：

```
16 Candidate Features
        ↓
Preprocessing
        ↓
SelectKBest(f_regression)
        ↓
Top 12 Features
        ↓
Multiple Linear Regression
        ↓
MAE / RMSE / R²
```

這樣老師要求的 **Linear Regression、Multiple Linear Regression、Feature Selection、10–15 個選定 Feature、Model Evaluation、CRISP-DM、預測圖、Prediction Interval、Kaggle Submission** 都已經包含。

有一個地方你執行時最可能需要修改：**實際 Kaggle CSV 的欄位名稱**。如果你把 `train.csv` 上傳給我，我可以直接依實際資料幫你把這支程式改成 **100% 對應資料集欄位、可以直接 Run All 的 `.ipynb` 版本**。

2026/10/15 下午12:05:52 · gpt-6-thinking